

The Open Source Vulnerability Intelligence Refinery

Access the complete, continually updated source inventory and ingestion schemas.

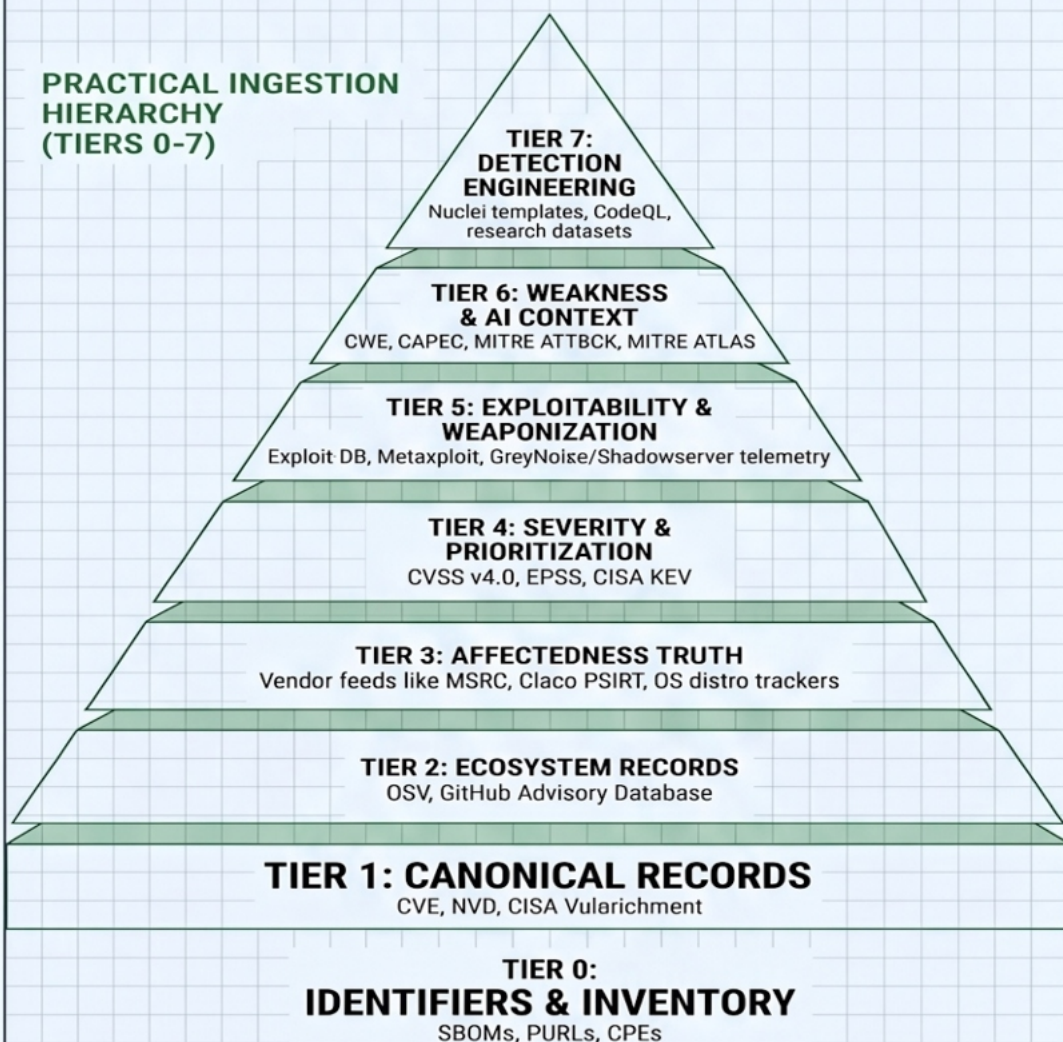
breachtrace.gitbook.io/vulnkeeper

CveToad Vulnerability Management Source Inventory Repository

CANONICAL INTELLIGENCE PIPELINE



PRACTICAL INGESTION HIERARCHY (TIERS 0-7)



CHAPTER A: INTAKE & IDENTITY

CVE record intake, schemas, governance

CHAPTER B: ENRICHMENT & DATA MODELS

Exploitability, prioritization, canonical data model

CHAPTER C: WEAKNESS & ATTACK MAPPING

CWE, CAPEC, ATT&CK, AI context

CHAPTER D: AFFECTEDNESS TRUTH

Packages, OSES, Vendors, Cloud, ICS/IoT

CHAPTER E: SUPPLY CHAIN EVIDENCE

SBOM, VEX, provenance, malicious packages

CHAPTER F: OPERATIONS & TELEMETRY

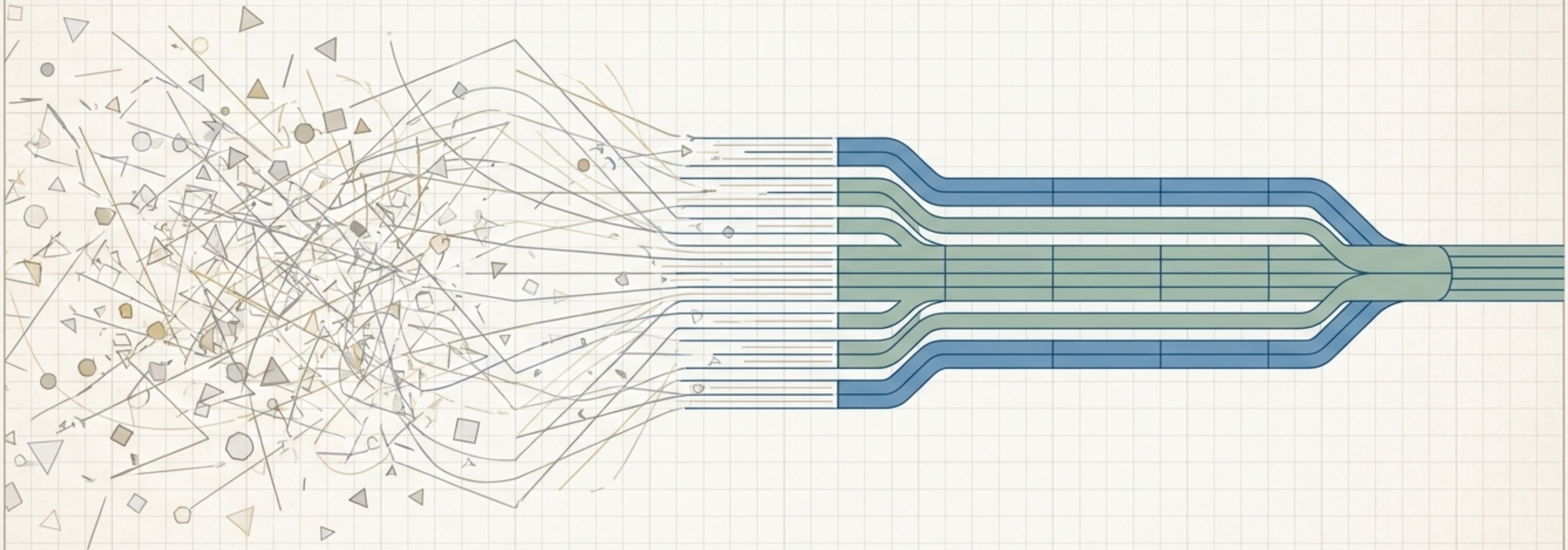
Automated detection, exposure telemetry, internet threat intel

CHAPTER G: GOVERNANCE & BASELINES

Compliance, software assurance, production baselines, operating model

Architecting the Vulnerability Intelligence Supply Chain

Transforming the global vulnerability catalog into a high-signal data engineering pipeline.



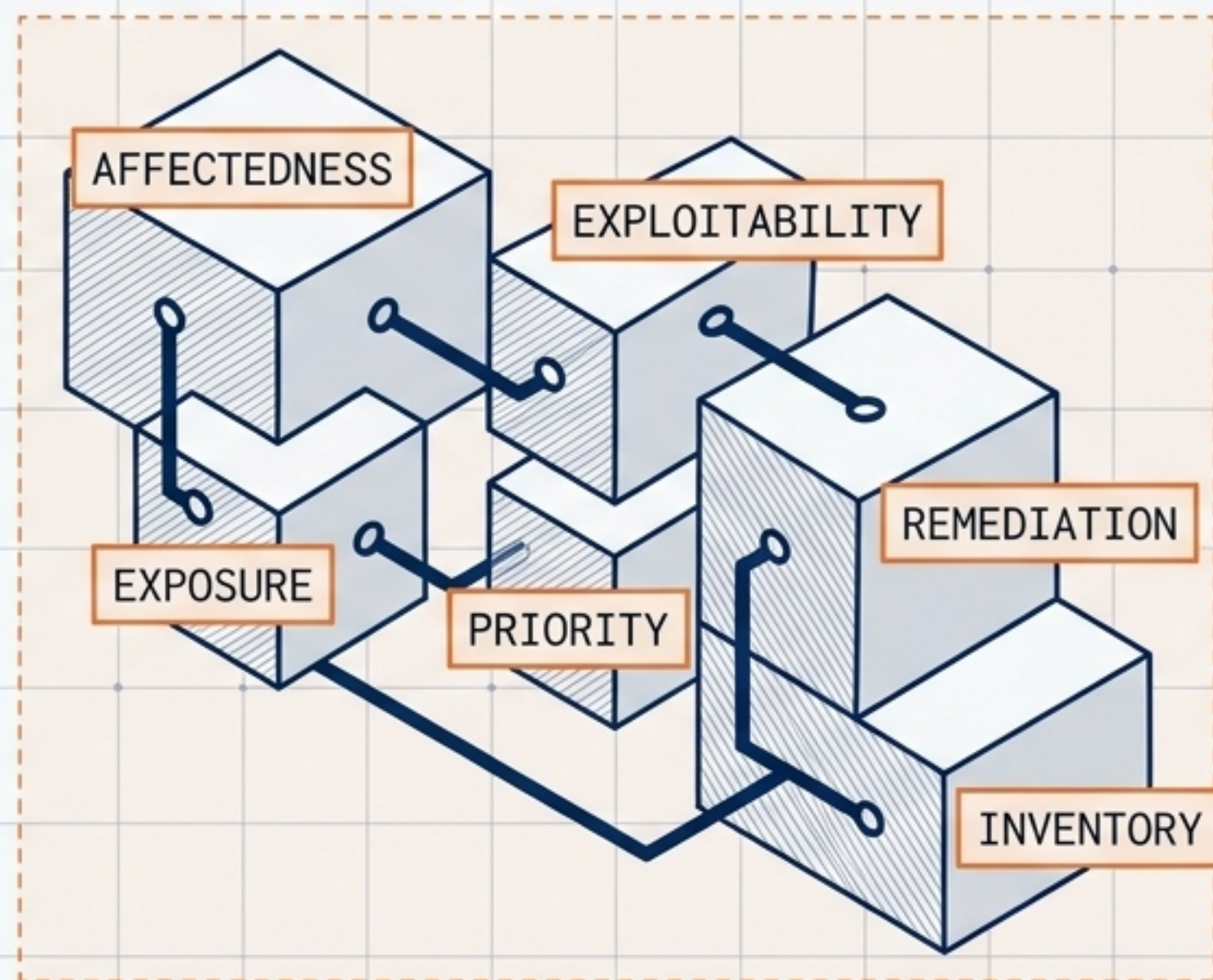
Based on the VulnKeeper / CveToad Inventory

THE VULNERABILITY REALITY GAP

No single source answers the fundamental operational questions:
Am I affected, exposed, exploitable, prioritized, & remediated?

CVE Record

Identity $\neq$ Affectedness. CVEs alone cannot reliably prove if your specific package version is actually vulnerable.



The Core Problem



The Problem

No single source provides a complete picture of impact, exposure, and remediation.

The Reality

A bare CVE identifies a record; it does not reliably prove affectedness. Distro feeds and ecosystem matching are required.

The Solution

Vulnerability management is a data engineering problem requiring an interlocking intelligence supply chain.

The Fragmentation of Vulnerability Truth

Question-to-Source Mapping

Identity: What is this flaw?
(Requires canonical schemas)

Inventory: Am I affected?
(Requires SBOM/SWID)

CVE-XXXX-XXXX

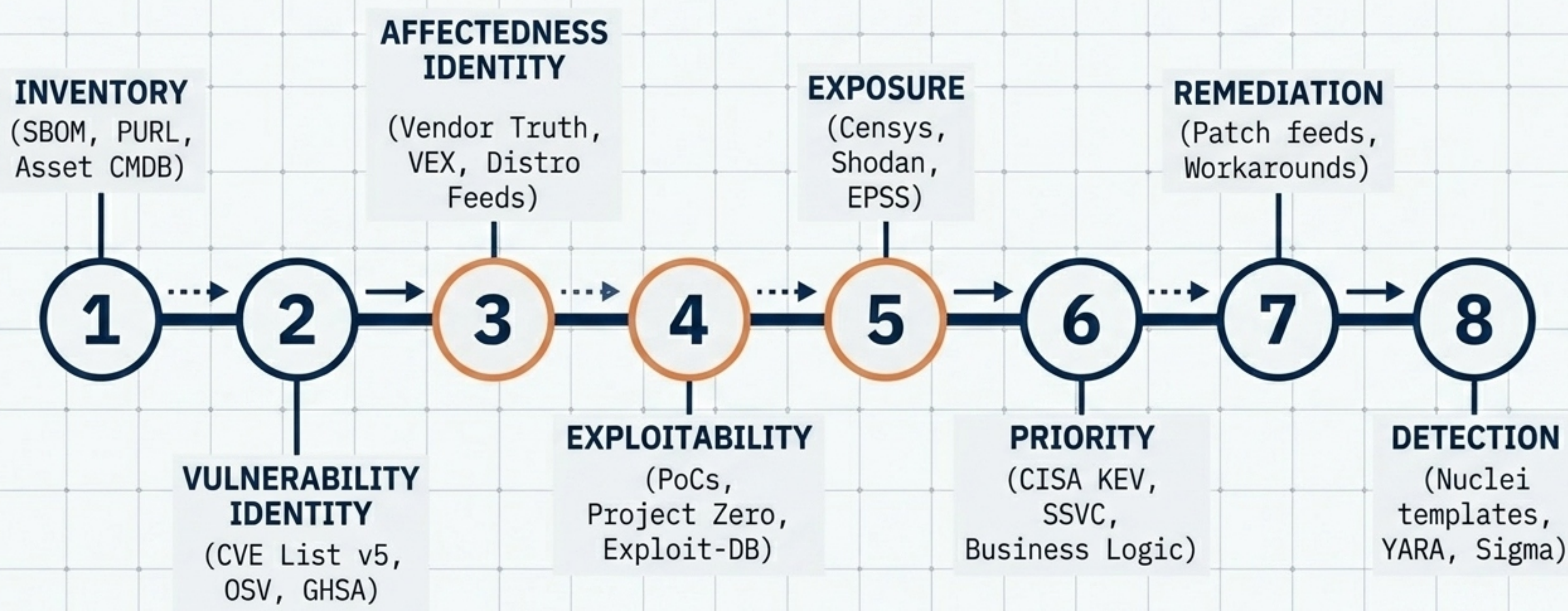
Affectedness: Is my
specific version patched?
(Requires OSV/Distro feeds)

Priority: How do I
sequence remediation?
(Requires SSVC)

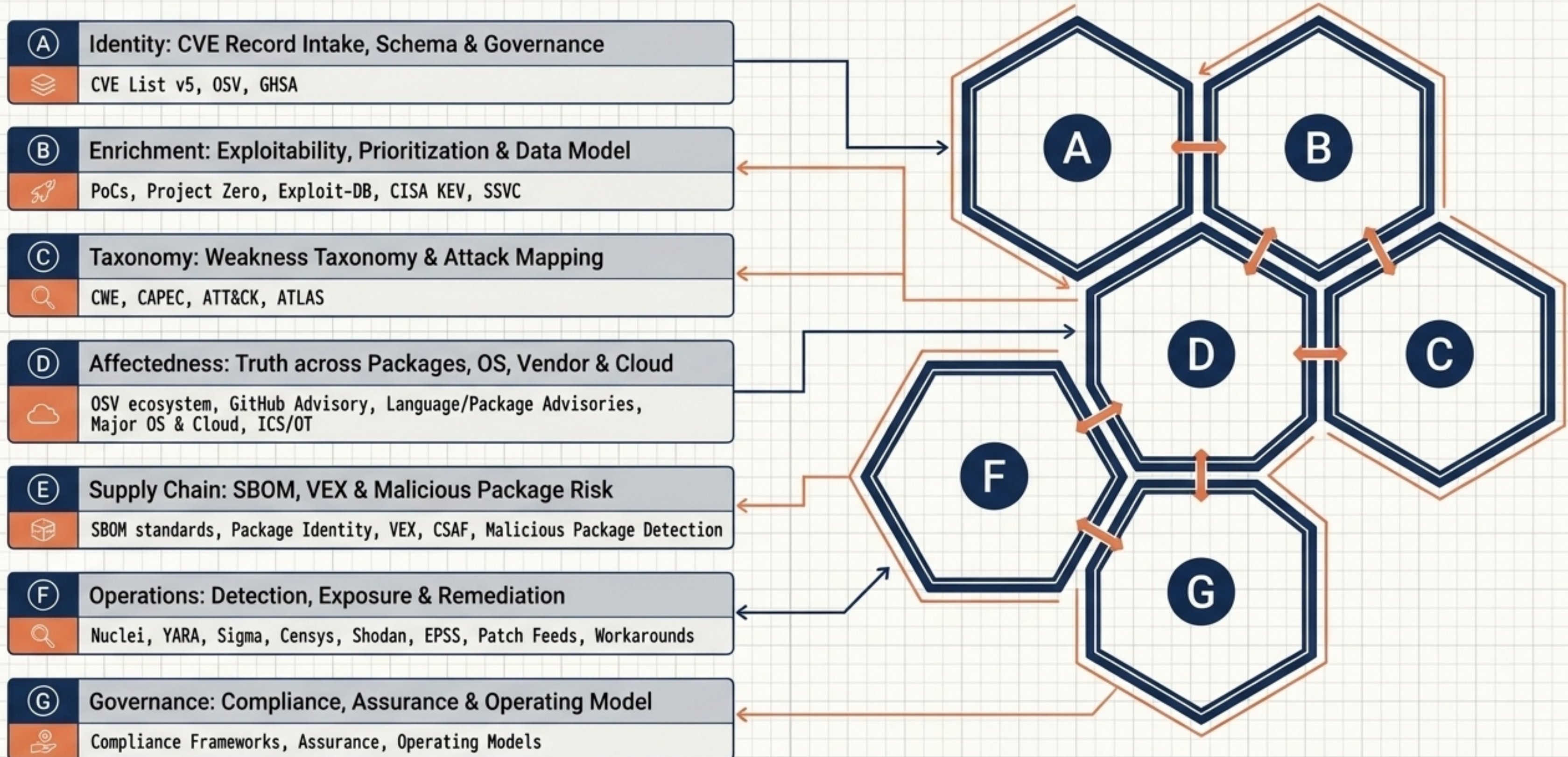
Exploitability: Is it
weaponized in the wild?
(Requires KEV/EPSS)

A CVE alone only identifies a record. NVD adds enrichment but lags or over-matches via broad CPEs. True operational intelligence requires mapping a fragmented supply chain of identities, affectedness, and telemetry.

THE CANONICAL PIPELINE FLOWCHART



The Vulnerability Lifecycle Operations

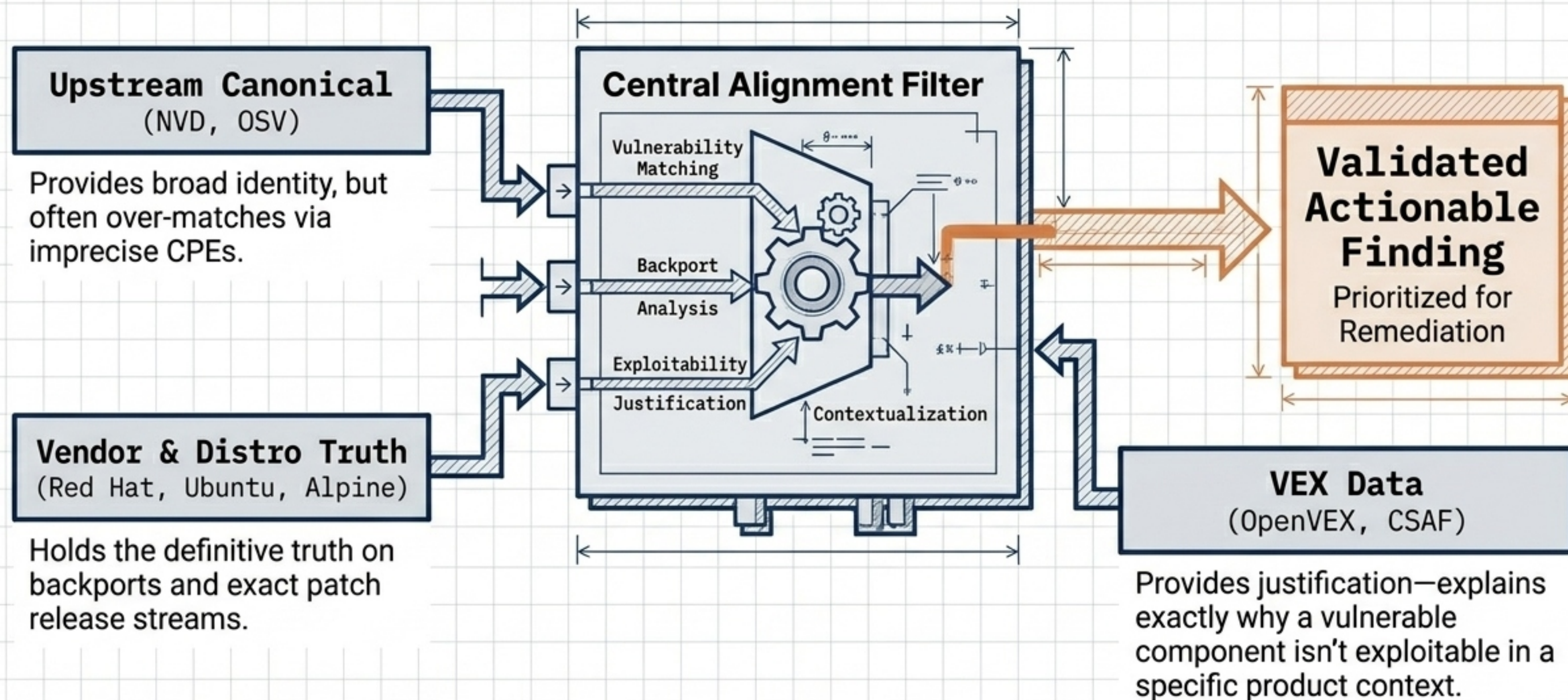


Connecting Structure to Operations

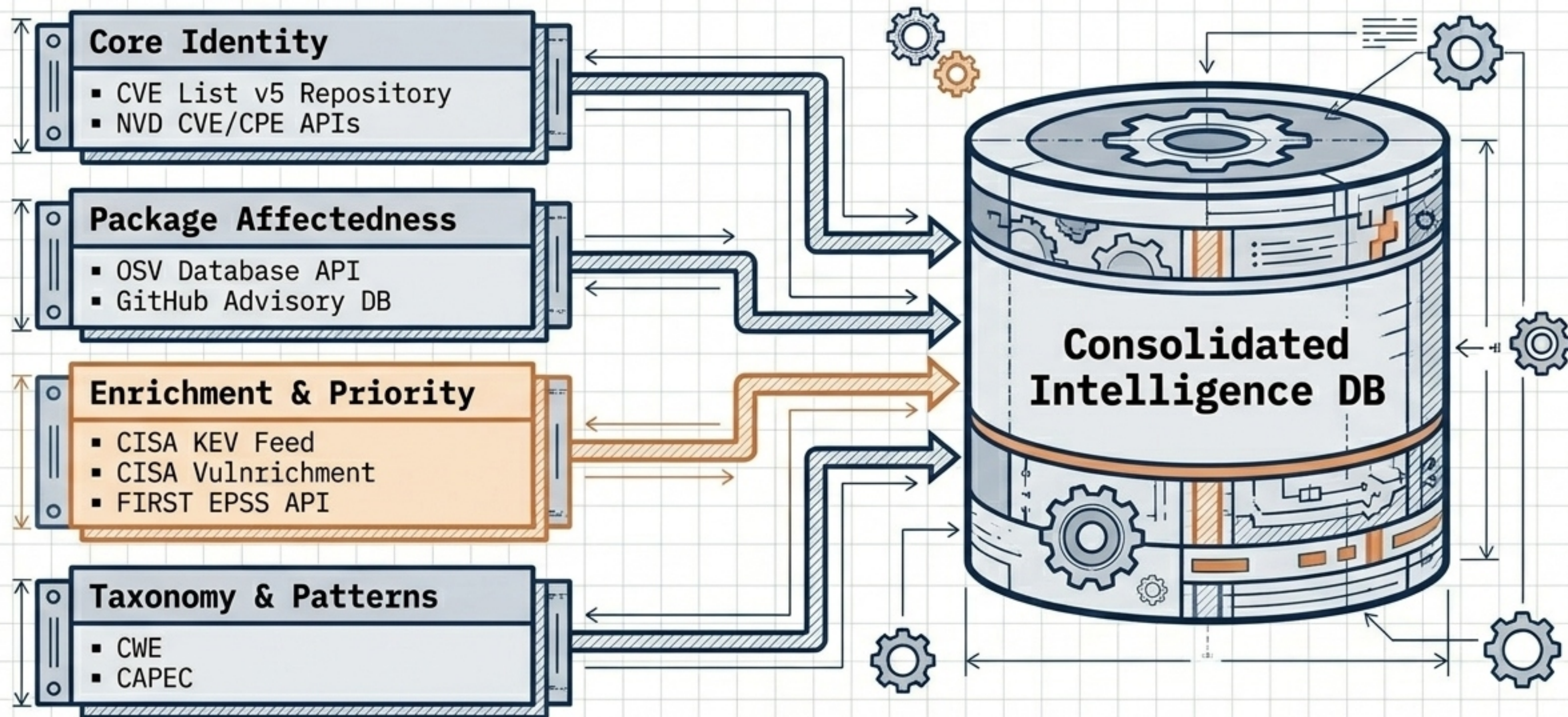
The structural data sources strictly govern downstream operational actions. You cannot prioritize without enrichment.

Y-Axis Chapter \ X-Axis	1. Inventory	2. Identity	3. Affected ness	4. Exploita bility	5. Exposure	6. Priority	7. Remedia- tion	8. Detection
👤 A. Identity	●	●	●	●	●	●	●	●
📄 B. Enrichment	●	●	●	●	●	●	●	●
📋 C. Taxonomy	●	●	●	●	●	●	●	●
📌 D. Affectedness	●	●	●	●	●	●	●	●
🔗 E. Supply Chain	●	●	●	●	●	●	●	●
>_ F. Operations	●	●	●	●	●	●	●	●
🏛️ G. Governance	●	●	●	●	●	●	●	●

Triangulating Affectedness Truth



The Minimal Production Source Set



Matrix: Vulnerability Intelligence Archetypes

Archetype	Primary Purpose	Format / Access	Key Constraint
Canonical (CVE/NVD)	Global identity & initial metadata	JSON APIs, GitHub mirrors	Often lags vendor disclosure; broad CPE matching.
Package Ecosystem (OSV/GHSA)	Open-source code and dependency truth	GraphQL, REST, Bulk ZIP	Version semantics vary wildly by language.
Vendor Feeds (MSRC/Distros)	Enterprise product patch availability	CVRF, CSAF, Custom APIs	Highly fragmented; requires complex scraping/polling.
Telemetry (KEV/EPSS)	Real-world exploitation prioritization	CSV, JSON feeds	Signals likelihood, not environmental reachability.

Prioritization Frameworks Compared

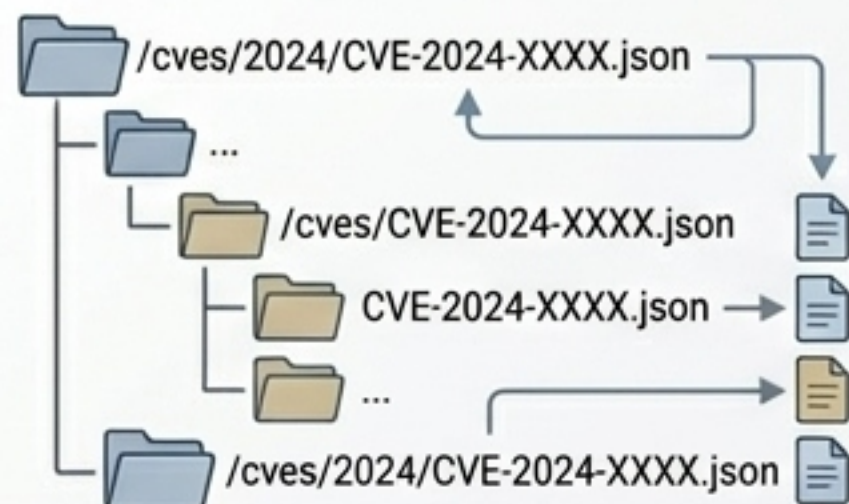
Framework	Measurement Focus	Operational Application
CVSS v4.0	Technical Severity	Baseline technical measurement. Warning: Do not use for exploit likelihood.
FIRST EPSS	Exploit Likelihood	Daily prediction of in-the-wild weaponization probability. Temporal and dynamic.
CISA KEV	Exploited Reality	Binary proof that a vulnerability is actively weaponized and exploited. Strongest operational signal.
CISA SSVC	Decision Logic	Tree-based model combining threat, asset context, and impact to mandate remediation timelines.

[Phase A] Identity, Intake & Schema

Establishing the canonical root of truth. What vulnerability is this?

The Canonical Mirror

CVE List v5 (GitHub)

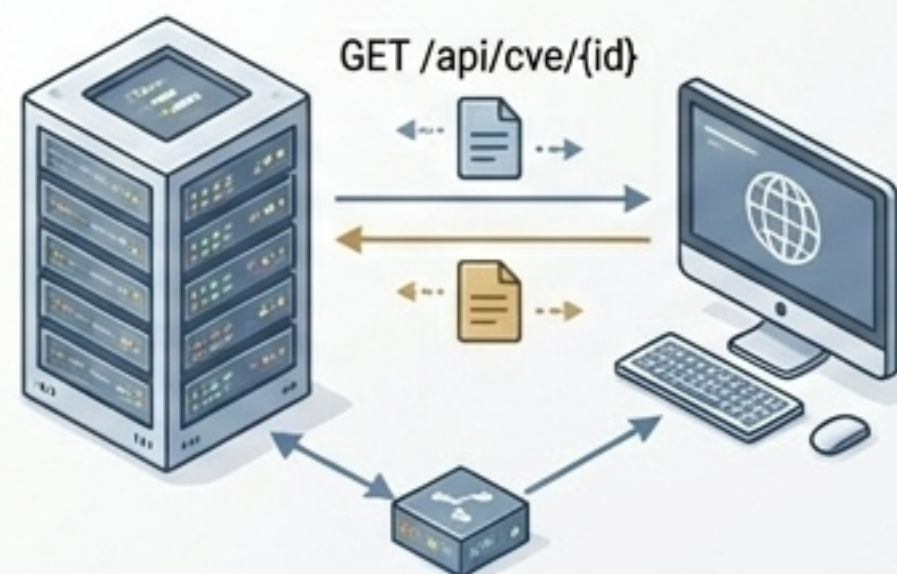


JSON Schema

The core cache for local mirroring and parsing.

Direct Retrieval

CVE Services API

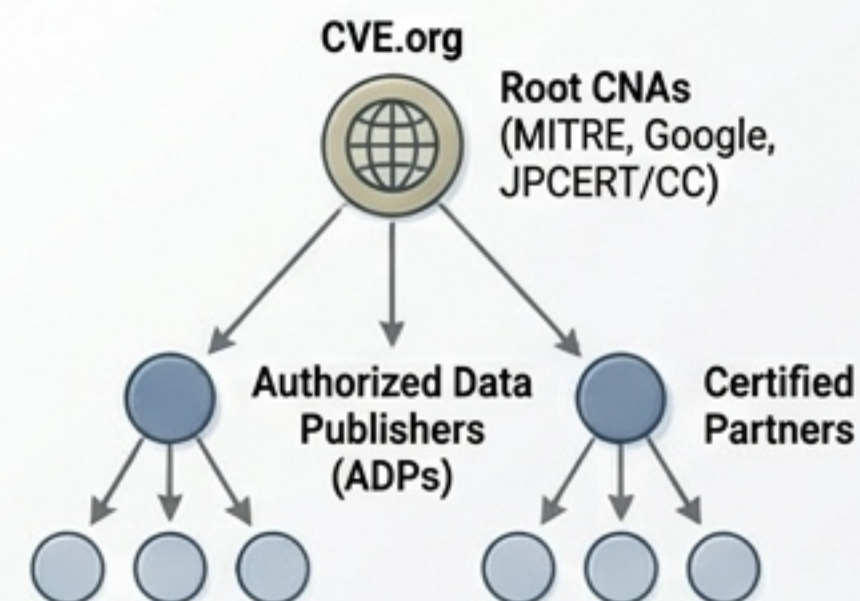


API

Primary endpoint: /api/cve/{id} for record lookup.

Program Governance

CNA / ADP Authority



Policy Governed

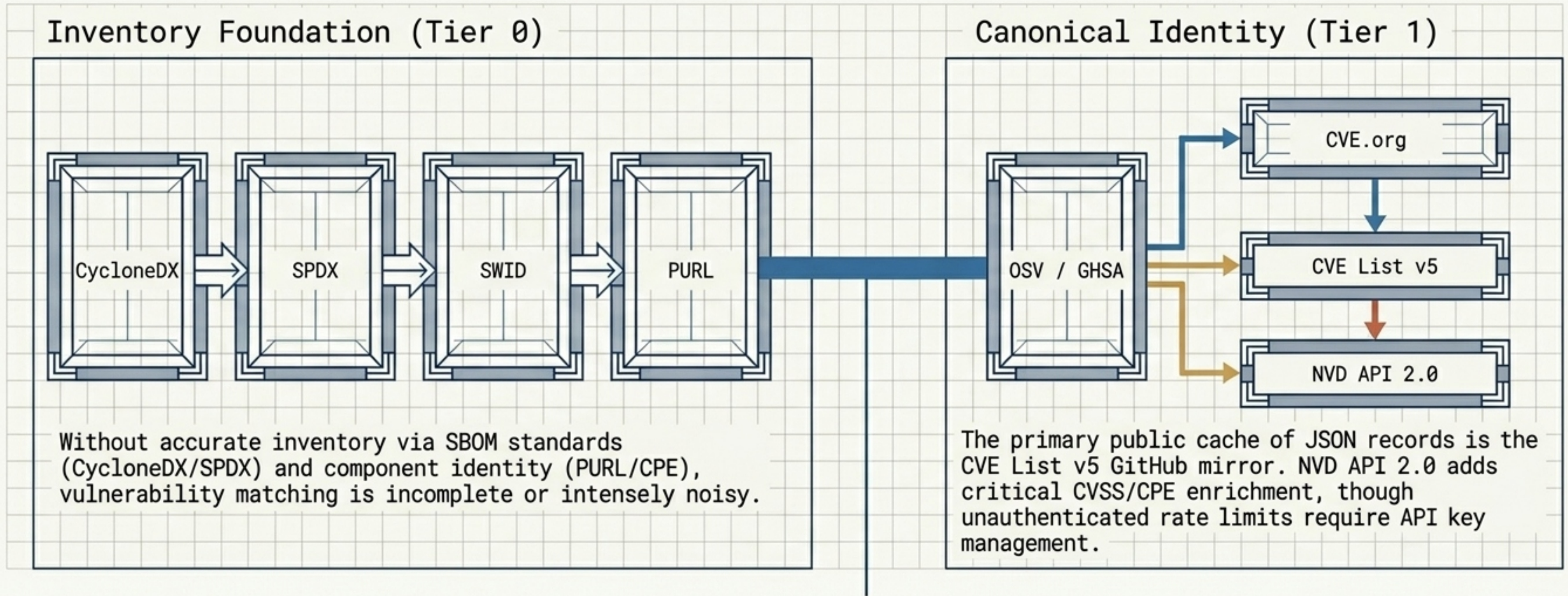
Root CNAs (e.g., Google, JPCERT/CC) assign initial records under CVE.org rules.

Free Public

Machine-Validated Schema

Rate Limits Apply

Tiers 0 & 1: You Cannot Patch What You Cannot See



Direct linkage: Package Identity
to Vulnerability Ecosystems

[Phase B] Enrichment & Prioritization

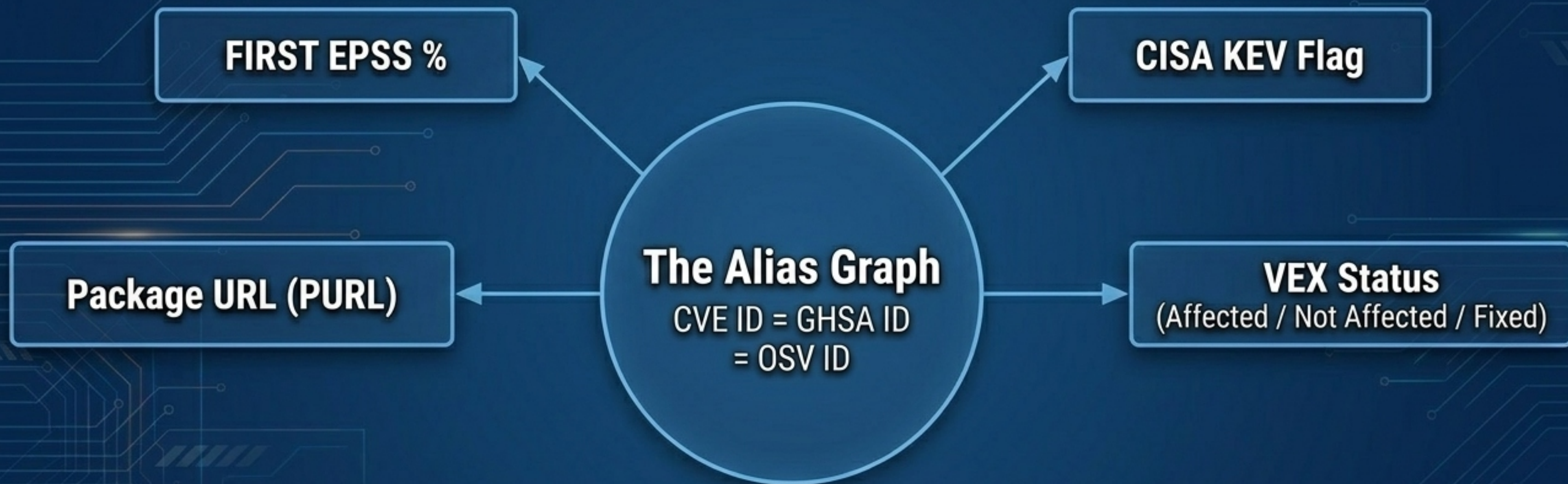
Adding severity, metadata, and exploit likelihood to the raw identifier.

Truth Hierarchy Matrix		
Baseline Context	Priority Signal	Exploit Reality
NVD Provides CVSS vectors, CPE matching, and references. 50 requests/30s with API key	EPSS & Vulnrichment FIRST EPSS predicts exploit likelihood via percentile ranking. CISA Vulnrichment adds SSVC decision points. Daily CSV/API updates	CISA KEV The authoritative catalog of actively exploited flaws. JSON schema validation required

Key Insight: Severity (CVSS) is not exploit likelihood (EPSS), and neither guarantees in-the-wild weaponization (KEV).

The Canonical Data Model Integration

How distinct sources communicate in a unified graph database.

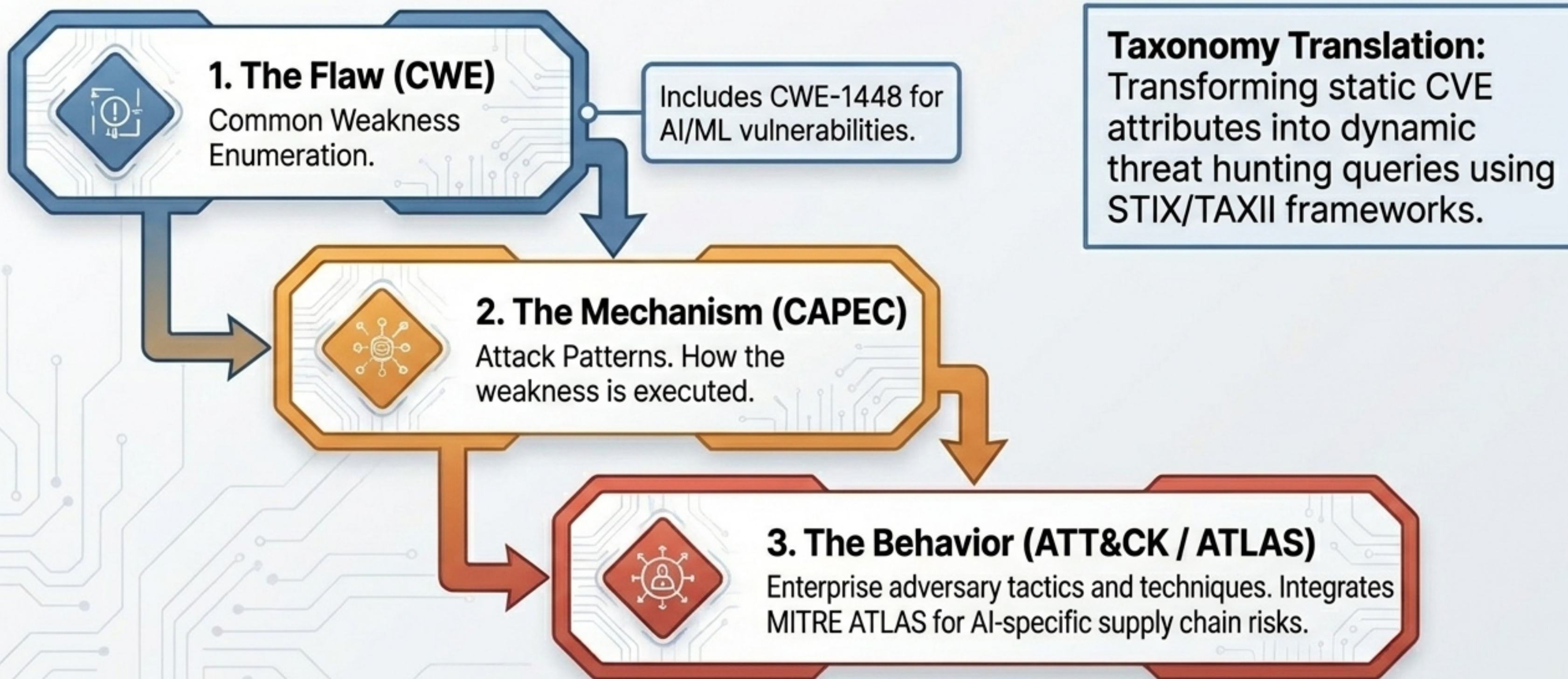


The Canonical Data Model:

Normalizing disparate identities into a single deal into a single relational alias graph.

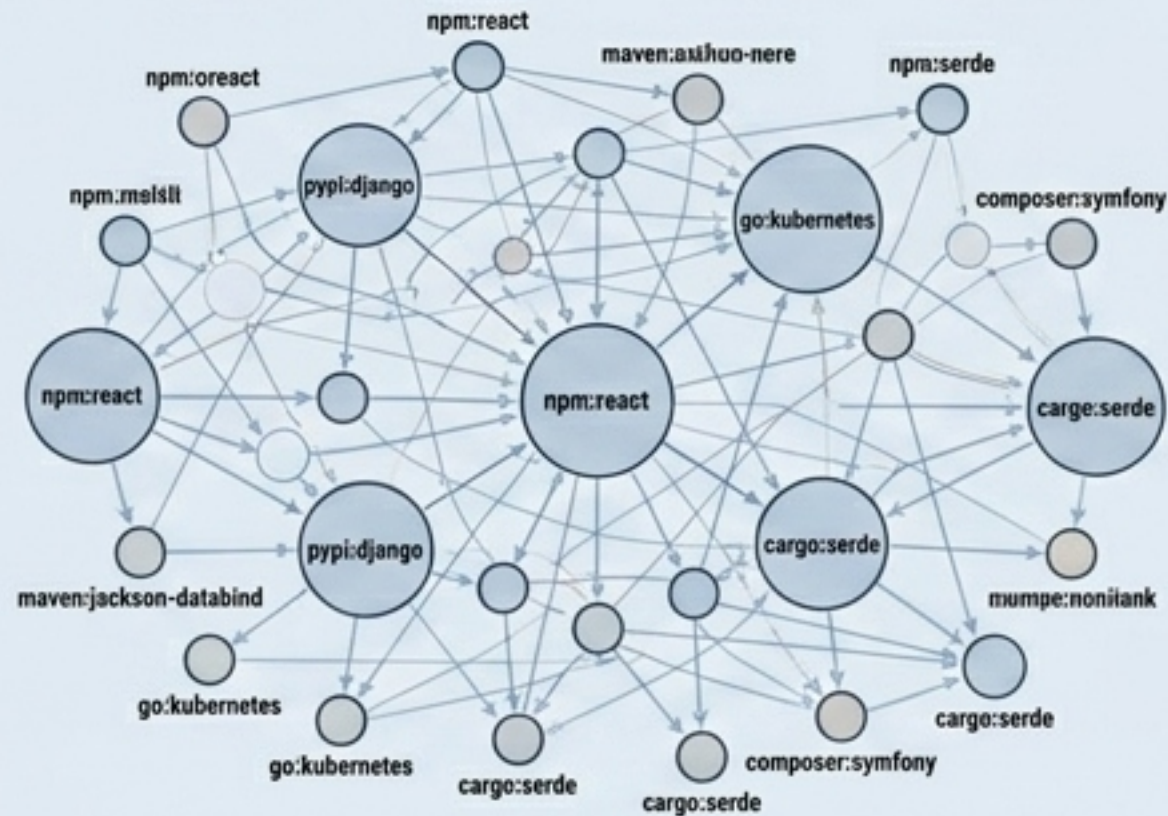
[Phase C] Taxonomy & Attack Mapping

Transforming static CVE attributes into dynamic threat hunting queries using STIX/TAXII frameworks.



[Phase D] Affectedness Truth

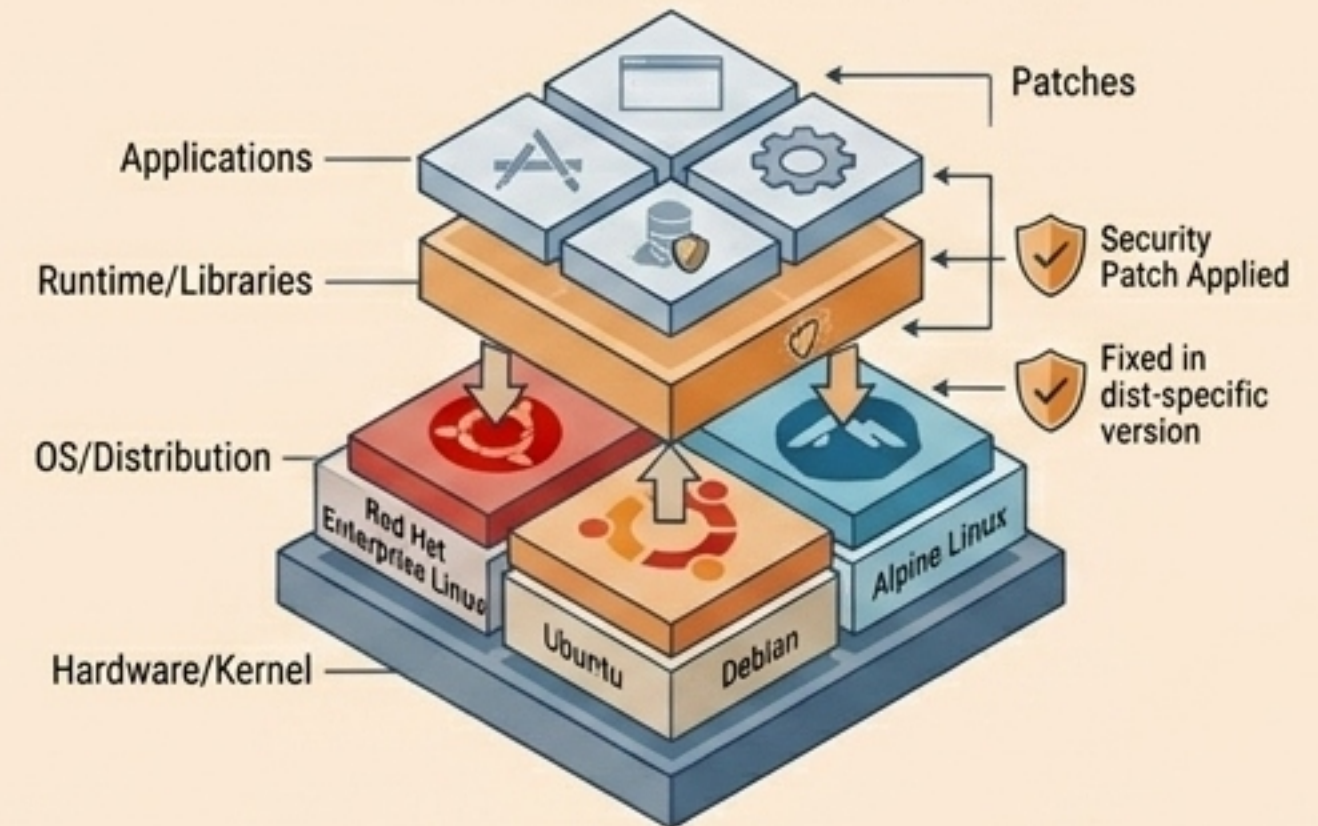
Package Ecosystems



Focus: OSV.dev & GitHub Advisory DB (GHSA).

- OSV and GHSA provide precise package-level matching.
- Delivers commit-level introduction and fix ranges.

Vendor & Distro Truth



Focus: Red Hat, Debian, Ubuntu, & Alpine SecDB.

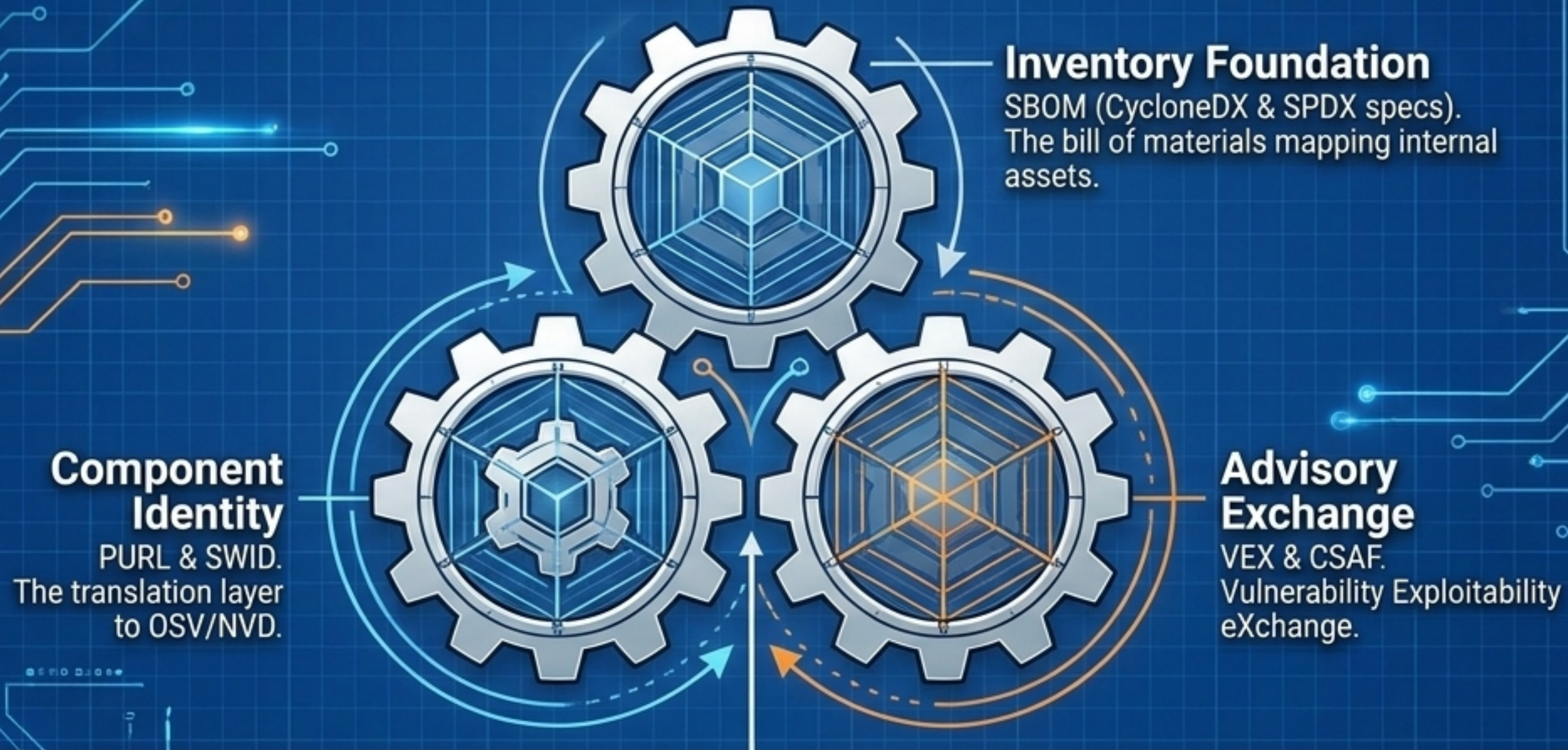
The Backport Problem

Upstream version strings may appear vulnerable, but distro-specific security patches negate the risk. Explicit vendor 'not affected' flags are mandatory ingestion points.

[Phase E] Supply Chain Evidence

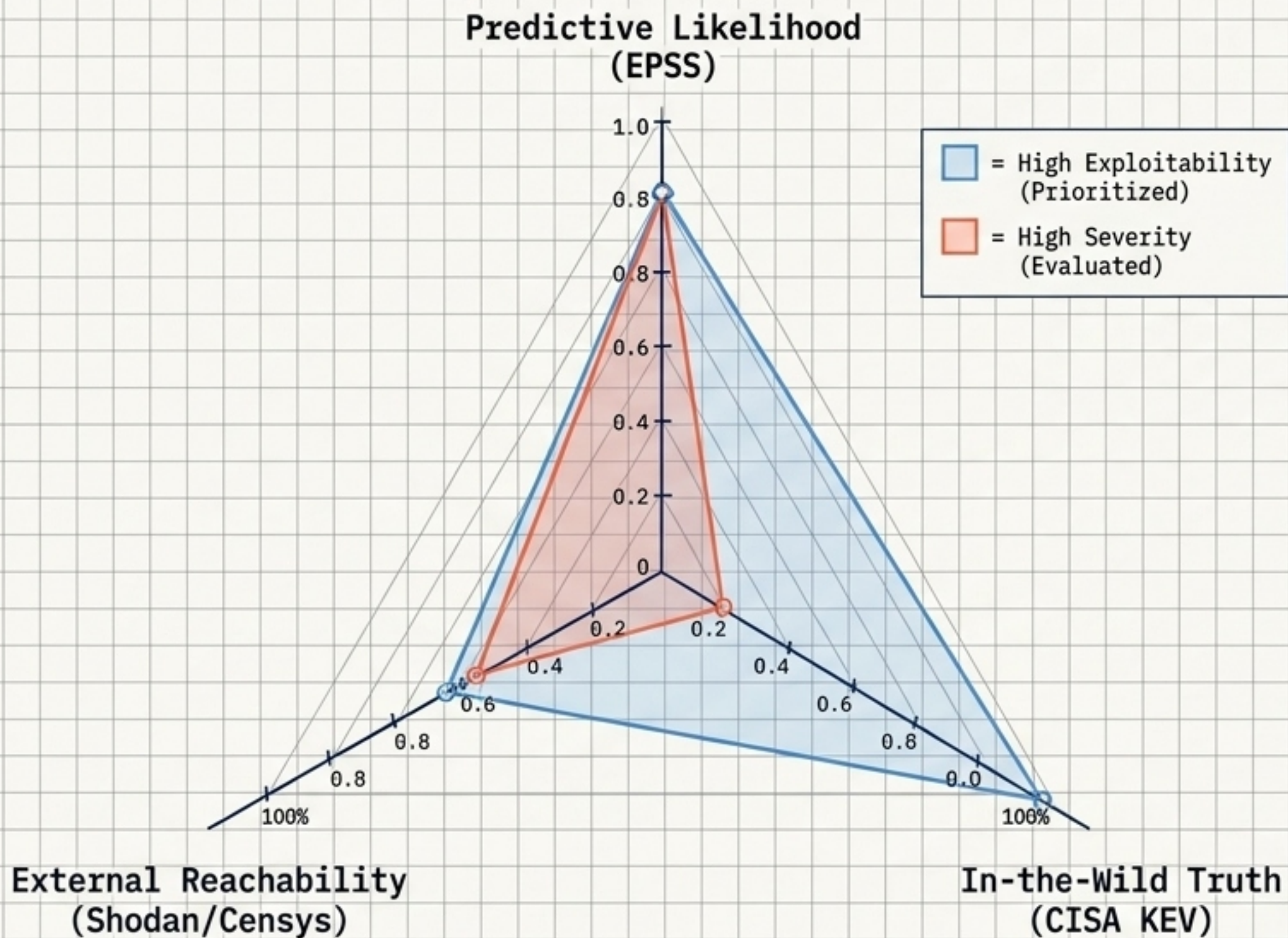
Binding vulnerabilities to actual internal inventory.

The Supply Chain Triad



The Power of VEX: Preserves the justification for a “not affected” state, providing auditability and instantly filtering out scanner false positives.

Steps 4 & 5: Severity ≠ Exploitability



In-the-Wild Truth

CISA KEV (Known Exploited Vulnerabilities) is the authoritative JSON catalog of vulnerabilities actively exploited by adversaries. It provides a strong, binary temporal signal.

Predictive Likelihood

FIRST EPSS provides daily probability-style exploit-likelihood percentiles, scaling prioritization across the backlog before active exploitation occurs.

Exposure Telemetry

Tools like Shodan and Censys map internet reachability, while GreyNoise visualizes scanning/exploitation noise versus targeted actor activity.

[Phase F] Detection & Exposure

Validating if the vulnerability is externally reachable or actively targeted.

Exposure Telemetry

- Shodan, Censys, LeakIX

Validating if the vulnerable asset is internet-facing.

Threat Intelligence

- GreyNoise, Shadowserver, VulnCheck

Filtering background scanning noise from targeted actor activity.

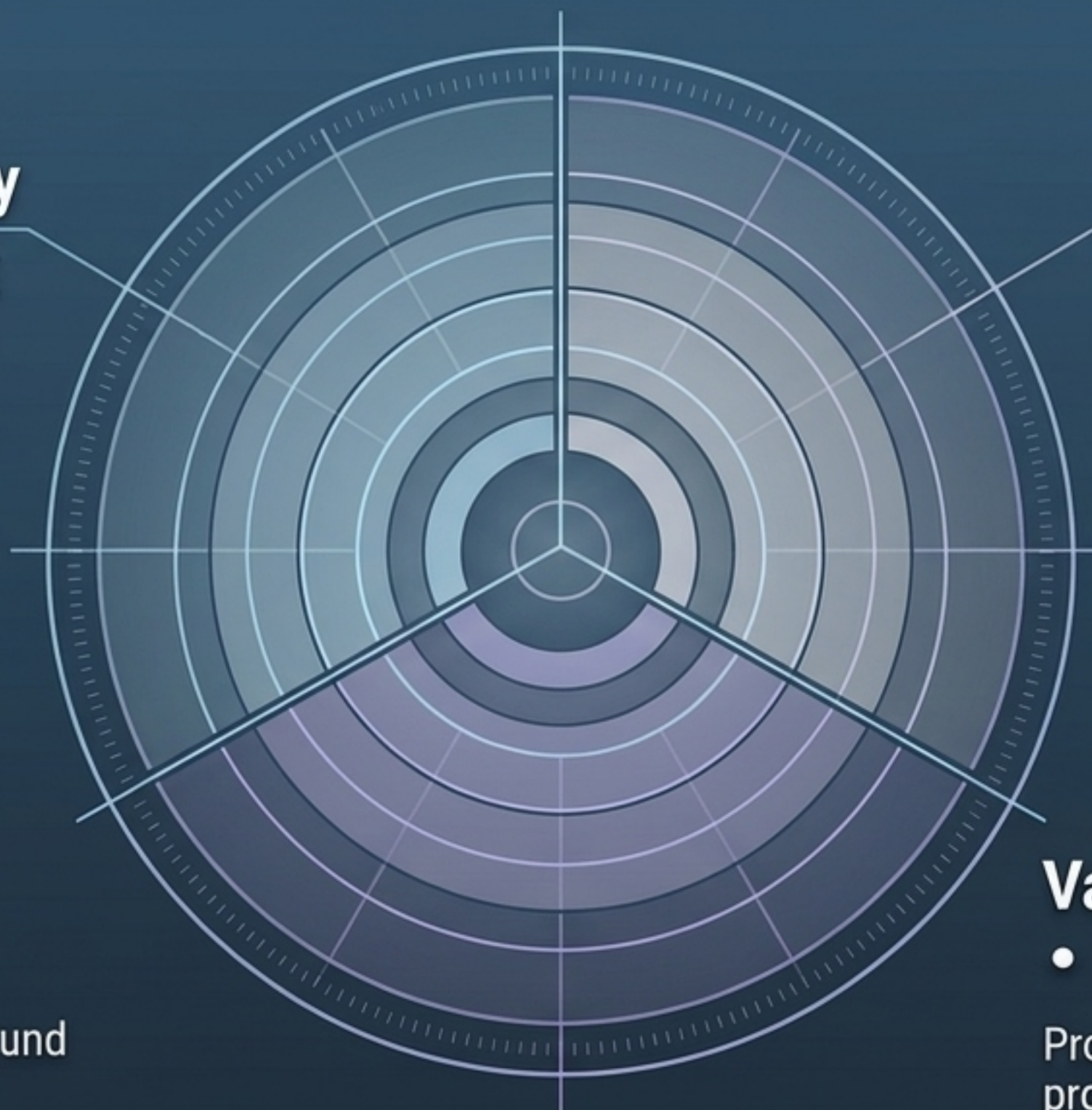
Validation & Signatures

- Nuclei, Exploit-DB, Metasploit

Proving the existence of weaponized proof-of-concept code.

Telemetry Context:

Distinguishing opportunistic background noise from targeted weaponization.



[Phase G] Governance & Baselines

Tying vulnerability metrics back to business risk and compliance standards.

Configuration Baselines

CIS Benchmarks & Controls. Essential for assessing if compensating controls mitigate a CVE.

Federal & Defense Standards

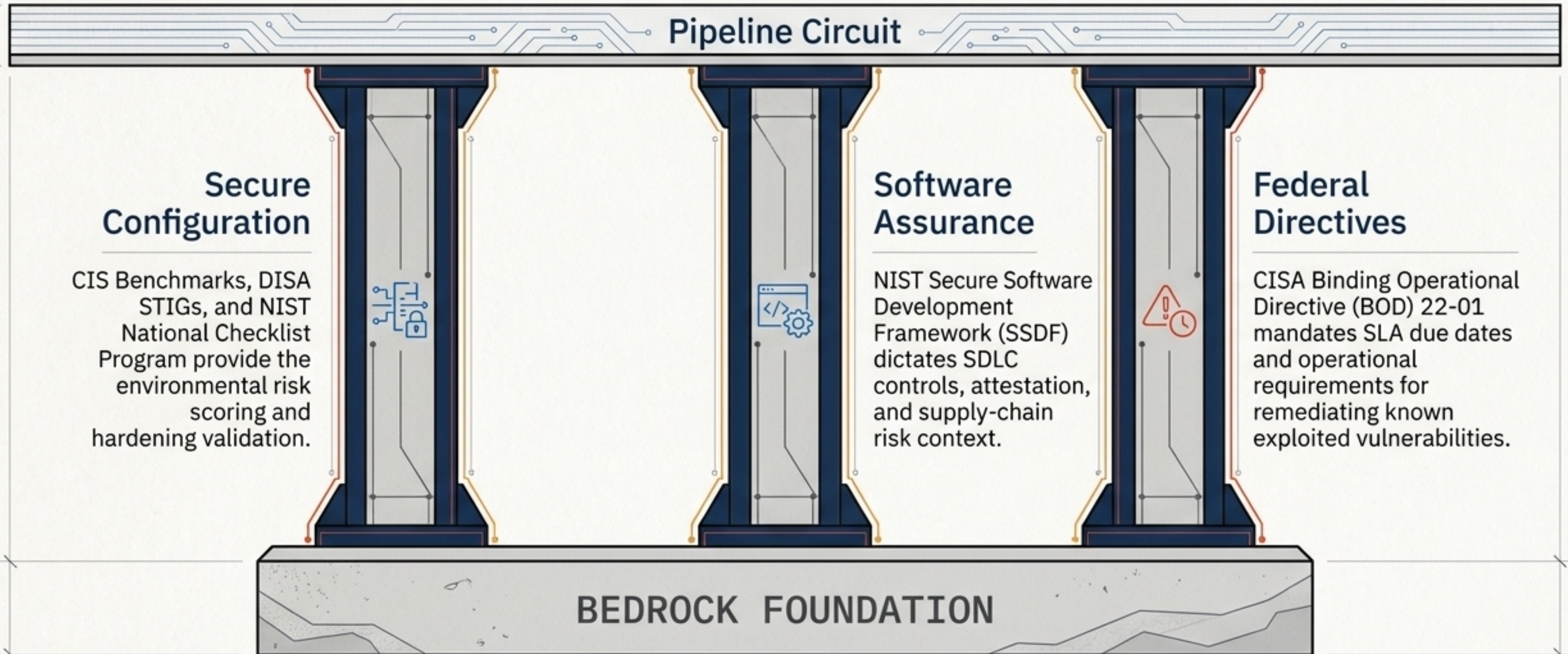
DISA STIGs and the NIST National Checklist Program (NCP).

Assurance Frameworks

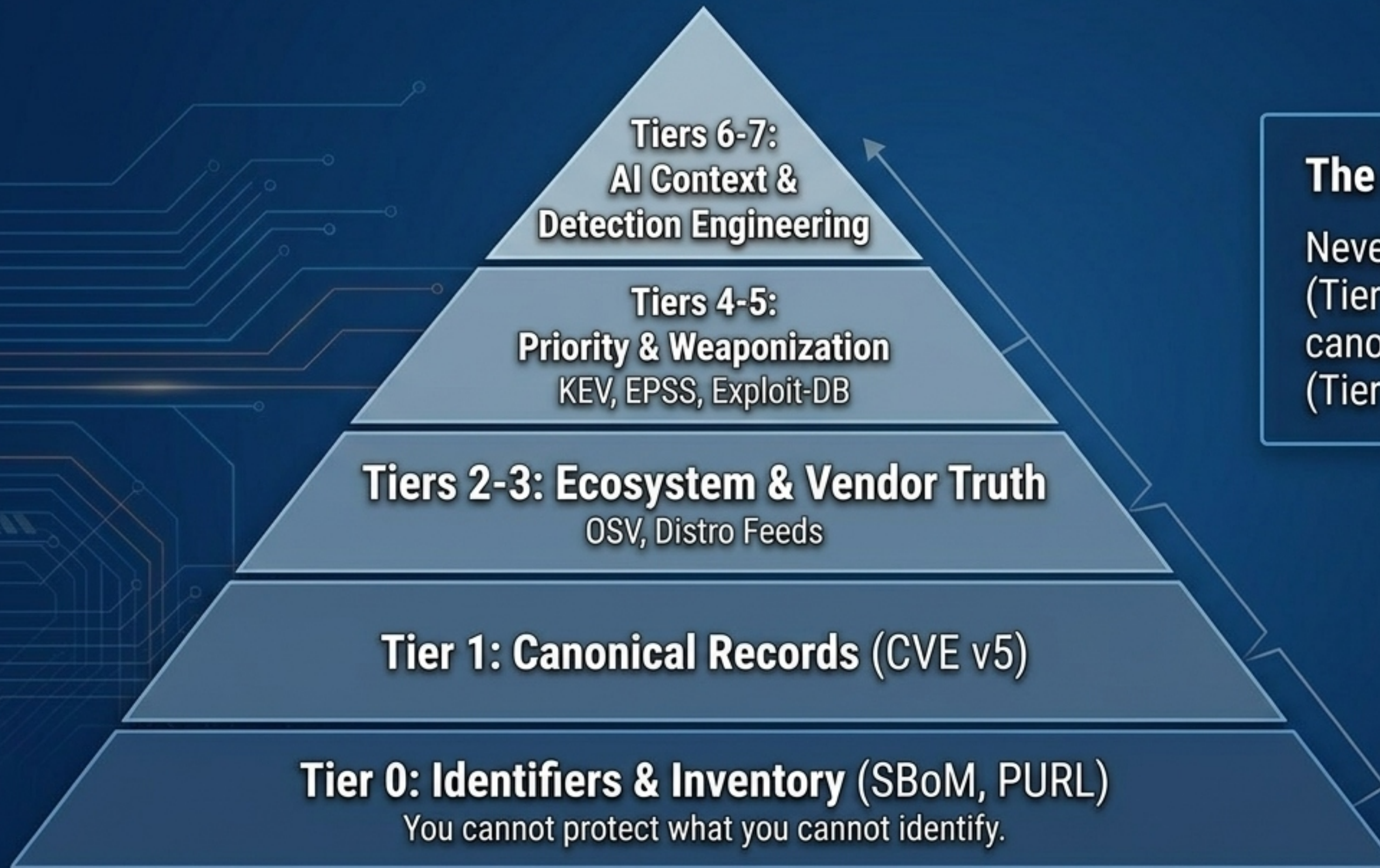
NIST Risk Management Framework (RMF) and AI-600-1 (Generative AI Profile).

Governance closes the loop: A vulnerability's final impact score is contingent on the baseline configuration and compensating controls of the host environment.

The Bedrock: Governance, Assurance & Compliance



Synthesis: The Ingestion Hierarchy



The Build Sequence:

Never ingest threat telemetry (Tier 7) before establishing a canonical inventory identity (Tier 0).

Synthesis: The Minimal Viable Production Set

If you cannot ingest the entire catalog, start here.

Data Phase	Critical Purpose	Minimum Production Sources
Identity	What vulnerability is this?	CVE List v5 (GitHub JSON)
Enrichment	What metadata exists?	NVD APIs + CISA Vulnrichment
Affectedness	What packages are vulnerable?	OSV.dev + GitHub Advisory DB
Vendor Truth	Is there a backport?	Distro Trackers (Red Hat, Ubuntu, Alpine)
Prioritization	Is it actually exploited?	CISA KEV + FIRST EPSS

The Minimal Viable Ingestion Engine:
Start here before expanding to the full 100+ source catalog.

The VulnKeeper Blueprint: Mapping the Vulnerability Intelligence Pipeline

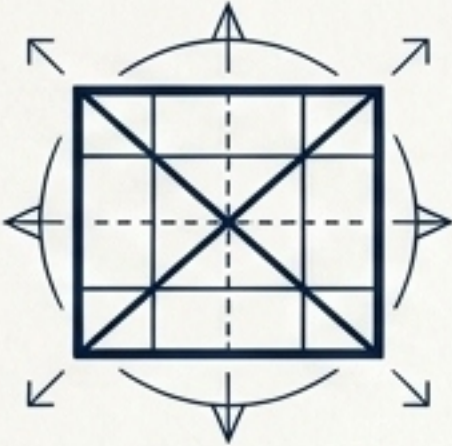
THE LIFECYCLE ECOSYSTEM (Chapters A–G)



THE CANONICAL VULNERABILITY PIPELINE



Operational Imperatives



1. Map the Pipeline First.

Establish your **inventory** and **canonical identifiers** before chasing **threat intel**. You cannot **patch** what you cannot bind to a **local asset**.



2. Accept Data Fragmentation.

NVD will never be a complete **oracle**. Realize that true **affectedness** requires joining **vendor feeds**, **distro backports**, and **package ecosystem data**.



3. Build the Minimal Stack.

Layer **predictive metrics (EPSS)** and **real-world telemetry (KEV)** over your **canonical truth** to automate **prioritization**.

Explore the comprehensive 100+ source inventory at the VulnKeeper GitBook.